

Tech Challenge Fase 2

POSTECH / FIAP

Aluno: Luis Felipe Martins da Silva

RM: RM372751

Desafio: Migração para Microsserviços na AWS

Repositório: github.com/luisfelipems/toggl MASTER phase2

Vídeo: youtube.com/watch?v=8nuwQRWMwDw

Dados do Aluno e Referências

Nome: Luis Felipe Martins da Silva

RM: RM372751

Repositório do Projeto:<https://github.com/luisfelipems/togglemaster-phase2>

Repositório de Apoio FIAP:<https://github.com/FIAP-TCs>

Vídeo de Apresentação:<https://www.youtube.com/watch?v=8nuwQRWMwDw>

1. Introdução

O ToggleMaster começou como um sistema monolítico na primeira fase do Tech Challenge. Era funcional, mas havia um grande problema: à medida que mais pessoas passaram a usar a plataforma, o sistema começou a ficar lento e difícil de manter. Toda vez que precisávamos mexer em uma funcionalidade, havia risco de quebrar outra. Era como tentar consertar um relógio enquanto ele continua funcionando.

Por isso, na Fase 2, nos foi dado um novo desafio: transformar esse monólito em um ecossistema de microsserviços. A ideia é bem simples: em vez de ter tudo em um único programa, dividir a responsabilidade em pequenos serviços especializados que se comunicam entre si. Cada um cuidando de sua própria tarefa, de forma independente e escalável.

O desafio envolvia não apenas reescrever o código, mas também provisionar uma infraestrutura robusta na Amazon Web Services (AWS) e orquestrar tudo usando Kubernetes (EKS). Era um exercício completo: da containerização ao deployment em um ambiente real de nuvem.

2. Procedimento de Criação dos Recursos

2.1 Os Cinco Microsserviços

Para começar, era preciso entender a estrutura. Recebemos o código-fonte de cinco microsserviços, cada um com uma responsabilidade bem definida:

- **Auth-Service (Go):** Responsável por gerenciar chaves de API e autenticação dos usuários. Usa PostgreSQL como banco de dados.

- **Flag-Service (Python):** Gerencia o CRUD das definições das feature flags. Também usa PostgreSQL.
- **Targeting-Service (Python):** Implementa as regras complexas de segmentação para determinar para quem cada flag deve estar ativa.
- **Evaluation-Service (Go):** É o serviço crítico — o "caminho quente" que retorna a decisão final (true/false) com altíssima performance. Usa Redis como cache.
- **Analytics-Service (Python):** Consome eventos de uma fila SQS e salva dados analíticos no DynamoDB, permitindo rastrear o uso das flags.

2.2 Containerização com Docker

O primeiro passo foi containerizar cada microsserviço. Isso significa criar imagens Docker para cada um deles. A ideia é simples: cada imagem empacotar o código, as dependências e tudo que a aplicação precisa para rodar, de forma isolada e reproduzível.

Criamos Dockerfiles para cada serviço, especificando as dependências necessárias (como bibliotecas Python ou módulos Go), configurando as variáveis de ambiente e definindo qual comando deve ser executado quando o container inicia. Era importante garantir que cada container escutasse na porta correta e tivesse acesso aos recursos que precisava.

2.3 Docker Compose para Ambiente Local

Antes de ir para a nuvem, precisávamos testar tudo localmente. Para isso, criamos um arquivo Docker Compose que orquestra todos os cinco serviços, além dos bancos de dados e caches necessários (PostgreSQL, Redis, SQS local e DynamoDB local).

O Docker Compose é como um "maestro" que coordena todos esses containers, criando uma rede entre eles para que se comuniquem. Foi essencial para testar a integração antes de fazer o deploy final.

2.4 Infraestrutura na AWS

Depois de validar localmente, chegou a hora de provisionar a infraestrutura na AWS. Isso envolveu várias etapas:

- **Cluster EKS:** Criamos um cluster Kubernetes gerenciado na AWS, que seria o "orquestrador" de todos os containers em produção.
- **Repositórios ECR:** Fizemos o push das imagens Docker que criamos para o Elastic Container Registry, que é o "registry" de imagens da AWS.
- **Bancos de Dados RDS PostgreSQL:** Provisionamos instâncias de PostgreSQL gerenciadas para os serviços que precisam delas.

- **ElastiCache Redis:** Configuramos uma instância Redis para o Evaluation-Service usar como cache de altíssima performance.
- **DynamoDB:** Criamos tabelas no DynamoDB para o Analytics-Service armazenar dados de análise em escala.
- **Fila SQS:** Configuramos uma fila SQS para o Analytics-Service consumir eventos assincronamente.

2.5 Configuração do Kubernetes

Com a infraestrutura AWS pronta, precisávamos configurar o Kubernetes para realmente orquestrar tudo. Isso envolveu instalar componentes adicionais e criar manifestos:

- **Metrics Server:** Instalamos o Metrics Server para que o Kubernetes conseguisse coletar métricas de CPU e memória dos pods.
- **Nginx Ingress Controller:** Instalamos o Nginx Ingress Controller para que o tráfego externo pudesse chegar até os serviços internos de forma controlada.

2.6 Deploy dos Microsserviços

Criamos manifestos Kubernetes (YAML files) que definem como cada microsserviço deve rodar. Esses manifestos especificam:

- Qual imagem Docker usar (da nossa ECR).
- Quantas replicas queremos (quantas cópias do serviço devem rodar em paralelo).
- Variáveis de ambiente e secrets (senhas, chaves de API, conexões com bancos).
- Limites de CPU e memória para cada container.
- Probes de saúde para o Kubernetes detectar automaticamente se algo está errado.

Fizemos o deploy de cada serviço, criando Services e Ingress rules para que pudessem ser acessados tanto internamente quanto externamente.

2.7 Escalabilidade com HPA

Um dos grandes benefícios de microsserviços é a capacidade de escalar. Configuramos o Horizontal Pod Autoscaler (HPA) para cada serviço, de forma que o Kubernetes automaticamente criasse mais cópias de um serviço quando o uso de CPU ou memória ultrapassasse certos limites. Isso garante que a plataforma continue responsiva mesmo com picos de carga.

3. Desafios Encontrados

Como em todo projeto de infraestrutura complexa, enfrentamos vários desafios pelo caminho. Felizmente, todos foram resolvidos, e aprendemos muito com cada um.

3.1 Problema: Erro 502 no Auth-Service

Logo após o deploy inicial, começamos a ver erros HTTP 502 (Bad Gateway) vindo do Load Balancer quando tentávamos acessar o auth-service. Isso é sempre um sinal de que o tráfego chega no serviço, mas algo está impedindo que chegue até o container de verdade.

3.2 Mismatch de Portas: 8001 vs 8005

Investigando os logs, descobrimos o problema: o Service do Kubernetes estava configurado para escutar na porta 8001, mas o container da aplicação Flask estava de fato escutando na porta 8005. O Kubernetes tentava rotear o tráfego para 8001, mas ninguém estava ouvindo lá — daí o erro 502.

A solução foi simples: ajustamos o Service para usar `targetPort: 8005`, redirecionando o tráfego da porta 8001 (a que o Load Balancer conhecia) para a porta 8005 (onde o serviço de verdade estava escutando).

3.3 Secret Incompleto do Banco de Dados

O Secret do Kubernetes que armazena as credenciais do banco de dados estava incompleto. Faltavam dois campos críticos:

- **DATABASE_URL:** A string de conexão com o PostgreSQL, no formato:
`postgresql://user:password@host:5432/dbname?sslmode=require`
- **MASTER_KEY:** O ARN da chave KMS (Key Management Service da AWS) usada para criptografar dados sensíveis.

Sem esses valores, a aplicação não conseguia nem iniciar. Atualizamos o Secret com os valores corretos e reiniciamos os pods.

3.4 Conectividade com RDS (Security Groups)

Mesmo com o Secret correto, havia problemas de conectividade com o banco RDS. O DNS do endpoint do PostgreSQL não estava se resolvendo corretamente dentro dos pods, e mesmo quando conseguia resolver, o TCP na porta 5432 não estava respondendo.

A culpa era dos Security Groups. Os Security Groups da AWS funcionam como um "firewall" entre os recursos. Os nós do EKS (que rodam os pods) e a instância RDS estavam em Security Groups diferentes e não havia uma regra de entrada permitindo a comunicação entre eles. Adicionamos a regra necessária: "permite TCP na porta 5432 vindo do Security Group do cluster EKS".

3.5 Permissões IAM para KMS

O auth-service também precisava usar a chave KMS para criptografia. Isso requer permissões específicas na AWS Identity and Access Management (IAM). A role IAM do pod não tinha permissões para:

- Descrever a chave KMS (`kms:DescribeKey`)
- Criptografar dados (`kms:Encrypt`)
- Decriptografar dados (`kms:Decrypt`)
- Gerar chaves de dados (`kms:GenerateDataKey`)

Adicionamos essas permissões à política IAM da role do pod, e o serviço conseguiu acessar a chave KMS sem problemas.

3.6 Resolução de DNS e Conectividade Final

Depois de resolver os problemas de Security Groups e permissões IAM, precisávamos validar tudo novamente. Testamos:

- Se o DNS do PostgreSQL se resolvia corretamente dentro do pod.
- Se a conexão TCP na porta 5432 respondendo.
- Se a conexão real com psql funcionava e conseguíamos fazer queries.
- Se os endpoints da API respondiam corretamente.

Depois de todas essas correções, o auth-service finalmente subiu de forma saudável e os erros 502 desapareceram.

4. Conclusão

O Tech Challenge Fase 2 foi um exercício muito prático e realista de arquitetura moderna em nuvem. Começamos com um monólito e o transformamos em um ecossistema de microsserviços, complete com containerização, orquestração Kubernetes e infraestrutura AWS escalável.

Os desafios encontrados — problemas de rede, configuração incorreta de portas, permissões faltando — são exatamente o tipo de coisa que engenheiros experientes enfrentam em projetos reais. Cada problema foi uma oportunidade de aprender como diagnosticar, investigar e resolver issues em ambientes distribuídos.

A maior lição foi entender que, em arquiteturas distribuídas, muitas coisas podem dar errado: DNS, Security Groups, permissões IAM, port mappings. É preciso ser metódico, testar em cada camada e entender como cada componente se comunica. Mas quando tudo está funcionando, você tem um sistema escalável, resiliente e fácil de manter — exatamente o que o mercado procura hoje em dia.

Obrigado pela oportunidade de enfrentar um desafio real. Foi uma experiência inolvidável!